Treasury Register -- Phase 2 Blueprint

Status: planning / understanding. **Audience:** product, engineering, treasury SMEs, the eventual first customer. **Purpose:** name every thing that is currently simulated in the demo, and describe exactly what it must become to be a real product a treasury team can rely on. No code changes yet -- this document is the map we build against.

Anchor assumption: every shape we have today stays. Table columns, JSON responses, the six-check gate, the audit-event contract -- none of these change. Phase 2 replaces the *source* of data, not the *shape* of it. That protects the 289 tests, the .context matrix and every downstream integration we build.

---

0. What is real in Phase 1 (the demo we already have)

We are not starting from zero. These behaviours are real and correct today, running end-to-end against seeded data:

- **Six-check gate on every write path** -- counterparty active, limit not breached, tenor within band, rating within band, concentration cap, group cap. Values-vs-limits captured in `check_run`. This is the control system.
- **Legal-entity scope** -- every user is scoped to one or more subsidiaries; scope-preflight refuses cross-entity writes.
- **Four-eye control** -- sensitive role grants require a second admin.
- **Deal lifecycle** -- propose -> approve -> instruct -> settle, with amendments that reverse accruals and repost cleanly.
- **FX hedging** -- expose -> check -> initiate -> link to underlying exposure.
- **Cash deployment planner** -- an optimiser producing three ranked plans within policy constraints.
- **AI advisory** -- model proposes, deterministic engine disposes. Every AI output re-runs through the six checks.
- **Audit trail** -- one writer, one table, every mutation captured.
- **Policy versioning** -- a decision made last quarter re-derives against the rules that applied then.
- **Compliance workspace** -- overview, audit trail, deal evidence, breach register, policy history.
- **Roadmap items shipped:** CORS (P0-03), rate limiting (P0-04), CI (P0-02), Postgres readiness (P0-01).

What isn't real: almost every input to those behaviours. That is what this document maps.

---

1. What is simulated today, and what it must become

Each row below is one *source of data* in the system. For every one, Phase 2 must specify: **where the data will come from, in what shape, at what cadence, and how it lands in our existing tables -- without changing those tables' shape.**

#	Data	Demo source today	Real Phase-2 source	Landing table (unchanged)
1	Counterparty master data	Seed rows in `counterparty`	Customer's KYC-approved counterparty list, exported from ERP or a treasury-master file	`counterparty`, `cp_limit`

#	Data	Demo source today	Real Phase-2 source	Landing table (unchanged)
2	Counterparty credit ratings	Free-text values in seed	Rating agency feed via Refinitiv/Bloomberg (Moody's/S&P/Fitch), refreshed nightly	`counterparty.rating`, version via `rating_action`
3	Rating changes / credit signals	Seeded `news_item` rows	Live feed -- GDELT (free) + a paid feed for enterprise; each item classified by counterparty	`news_item`
4	Legal entities	3 seeded subsidiaries	Customer's group structure, one-off setup + change control	`legal_entity`
5	Currency exposure (AR/AP + forecast)	Static seeded rows	Nightly extract from ERP (AR ageing, AP forecast, sales forecast overlay)	`currency_exposure`
6	Cash position (bank balances)	Not modelled; deals implicitly assume unlimited cash	Daily balance sweep from banks -- MT940 or ISO 20022 camt.053	New `cash_balance` table (Phase 2 addition -- shape TBD, additive only)
7	FX spot rates	Deterministic quote engine (algorithmic, not real)	Bloomberg/Refinitiv/Xignite market data snapshot, cached with a TTL	New table (additive), or `market_data_snapshot` read-through cache -- decide in Phase 2 kickoff
8	FX forward rates	Same deterministic engine	Same feed as spot; forwards are quoted or derived from swap points	Same as above
9	Interest-rate benchmarks (SONIA, EUR STR, SOFR)	Not modelled	Central-bank publications + market data provider	New reference table (additive)
10	Deals proposed by treasurers	Web UI (real)	Web UI (unchanged) + optional API from an OMS if the customer uses one	`deal`
11	Deal approvals	Web UI approve button	Web UI (unchanged) -- a real signer clicking approve is a real signal	`deal.approved_by`, `audit_ev`

#	Data	Demo source today	Real Phase-2 source	Landing table (unchanged)
12	Bank confirmations (post-trade)	Seeded MT320-shaped rows	Real SWIFT confirmations -- MT320 for money markets, MT300 for FX, camt.053 for statements	`confirmation`
13	Statement reconciliation	Fake statement lines	Bank statement feed -- MT940 daily / camt.053 near-real-time	`statement_line`
14	Payment instructions (outbound)	Written to `deal_instruction` table	Real SWIFT MT202 / pain.001 outbound; response codes captured; kill-switch on failure	`deal_instruction`, `bank_ack` (additive)
15	Accounting postings (outbound)	Written to `accrual` table only	ERP GL posting -- Oracle Financials / SAP / NetSuite via the connector; two-way ack	`accrual` unchanged; new `gl_posting_ack` table (additive)
16	Users + roles	5 seeded users, password-based	Corporate IdP -- Entra ID / Okta / Auth0 via OIDC; roles mapped from IdP groups	`app_user`, `membership` uncha new `sso_identity` table (additive)
17	Policy versions	Seeded, plus /admin/policy editor	Same UI; the four-eye write control lands with the editor (ADR-0017 called this out)	`policy_version`
18	Audit trail	Written by `audit_service`	Same. Retention + archive policy added; compliance data residency confirmed	`audit_event` -- retention job is new work, not shape change
19	AI advisor activity	Groq call in-flight, no persistence	Same call + new `ai_run` table capturing prompt, response, model, tokens, fallback flag, linked decision (closes Q-004)	New `ai_run` (additive)
20	Email delivery	Console log	Real SMTP -- AWS SES / Azure Comms / SendGrid -- for invite, breach alert, approval request	No new table; `activation_token` unchanged

Every table on the right stays. Nothing is deleted. New tables are additive and never break an existing query.

2. Data source map -- per domain, in detail

Each domain below answers three questions: **where does the data come from, how does it get handled, and who uses it.**

2.1 Counterparty master data + credit limits

Today: 6–8 seeded counterparties (Meridian, Caledonia, Barclays, Harbour, Nova, NTS...). Every field is fictional.

Phase 2 source: the customer's KYC-approved list. Practically:

- **Initial load** -- one-off import from the customer's existing treasury system export (usually CSV or their ERP's counterparty extract). Includes LEI (Legal Entity Identifier), rating, sector, jurisdiction, group parent, limit amount.
- **Ongoing changes** -- admin UI is the write surface. Every change is versioned (`counterparty_verification`, `cp_limit`). No cron-poll of an external source unless the customer has a master-data-management platform (unusual outside banks).
- **LEI lookup** -- GLEIF public API (`api.gleif.org`) for validation on entry. Free, unauthenticated, reliable.

Handling: 1. CSV goes to a staging table. 2. Validator runs: LEI checksum, jurisdiction ISO codes, currency codes, limit >= 0, rating in known catalogue. 3. A CFO approves the import (this is a policy decision, not a data decision). 4. Rows land in `counterparty` + `cp_limit` with a full audit event per row.

Used by: every six-check run (`counterparty_active`, `limit_headroom`, `concentration`, `group_cap`). Never mutate a row that is referenced by a check_run -- supersede + record.

Failure mode: import fails validation -> rollback whole batch, one audit event `counterparty.import_rejected`. Nothing lands.

Owner: customer's treasury operations lead, initially. Move to admin persona after week 1.

2.2 Credit ratings + rating actions

Today: ratings sit as free text ("A", "BBB+") in the counterparty row. No history.

Phase 2 source: rating agency feeds. The realistic options:

- **Free tier / news-based** -- GDELT + Wikidata for public downgrades. Latency is hours to a day. Fine for a starter product.
- **Paid tier** -- Refinitiv or Bloomberg. Sub-second delivery of any rating action. Cost: $$$. Only when the first customer explicitly demands it.
- **Direct** -- S&P / Moody's / Fitch APIs. Requires a licensing deal.

Recommendation: start with GDELT + a nightly ETL that reads the counterparties we care about; add a paid feed adapter behind a `RatingProvider` port when the customer's contract says so.

Handling: 1. Nightly job pulls rating actions for our watchlist (every counterparty we hold a limit against). 2. Each action becomes a `rating_action` row + an `audit_event(action=counterparty.rating_updated)`. 3. If the new rating breaches the counterparty's tenor band, the nightly job **auto-raises an exception** in `exception_item` and pings the compliance officer. 4. UI shows the change with a `TrendChip` in the counterparty detail.

Used by: rating band check (`rating_band` policy table), tenor cap check, concentration recalculation.

Failure mode: feed unavailable -> nothing changes, alert fires. Never silently roll a stale rating forward.

2.3 News / credit signals

Today: seeded `news_item` rows describing hypothetical events.

Phase 2 source:

- **GDELT 2.0** -- global news event database, near-real-time, free. Query by entity name.
- **Optional: Refinitiv NewsScope / Bloomberg** -- commercial, higher signal.

Handling: 1. Poller runs every 15 minutes. 2. Each news item is scored (sentiment, entity match, severity) -- this is where the AI advisory layer earns its keep: an LLM classifies free-text news into `{routine, watch, action}` for a *specific* counterparty. The classification is deterministic-fallback-backed exactly like the existing FX narrator. 3. `news_item` row + audit event `news.received`. 4. If the score is `action`, an exception is queued and a treasurer sees it on the ticket surface.

Used by: the treasurer's counterparty card (already there in the demo -- will start showing real news), and the advisory layer as an input feature.

Failure mode: poller silent for > 1 hour -> paging alert. Treasury cannot make decisions on stale credit intelligence.

2.4 Currency exposure (AR + AP + forecast)

Today: static seed rows per currency.

Phase 2 source: Oracle Fusion Cloud ERP -- specifically Payables (AP), Receivables (AR), and EPM Cloud (cash forecast) -- accessed through the **Treasury Register Connector** (see §3.3). This is the single biggest integration in the product.

How it flows in:

1. **Scheduled pull, not push.** The connector calls Oracle Fusion Cloud REST every night at 02:00 local for each customer legal entity: - `GET /fscmRestApi/resources/latest/invoices?q=BusinessUnit='<LE>';InvoiceDate BETWEEN '<from>' AND '<to>'` (AP) - `GET /fscmRestApi/resources/latest/receivablesInvoices?q=BusinessUnit='<LE>'` (AR) - EPM Cloud data-form pull for cash forecast (next 90 days by currency)
2. **Transformation** happens inside the connector -- pivots Oracle rows into our `currency_exposure` shape: `(legal_entity_id, currency, gross_receivable_pence, gross_payable_pence, forecast_pence, as_of)`. Oracle's `Amount` is a decimal; we multiply by 100 and round for pence-integer storage.
3. **Load** -- insert new snapshot; mark prior snapshot superseded. Never mutate a snapshot already referenced by a `hedge_link` -- hedge decisions carry the exposure they were made against.
4. **Mapping table** -- `BusinessUnit` in Oracle -> `legal_entity_id` in our schema. Per-customer mapping in a new `oracle_entity_mapping` table (additive).

Used by: FX hedging, planner, exposure summary, compliance overview.

Failure mode: Oracle unavailable or file empty -> the ETL alerts if the last successful snapshot is more than 24h old, AND the FX summary UI shows the `as_of` timestamp with an amber banner. Never silently roll forward.

2.5 Cash position (bank balances)

Today: not modelled at all. The planner assumes cash exists.

Phase 2 source: banks. Two protocols:

- **SWIFT MT940** -- end-of-day statements, one file per bank per account, delivered via SWIFT FIN.
- **ISO 20022 camt.053** -- modern replacement, richer, delivered via SWIFTNet FileAct or bank host-to-host.

How it flows in: 1. Bank sends the file to a bank connectivity provider (BankLine, Kyriba's connectivity module, Fides, or the customer's SWIFT service bureau). 2. Provider drops the file in our sFTP inbox or webhook. 3. Parser extracts opening + closing balance + movements per account. 4. Landing in new table `cash_balance` (additive). 5. Reconciliation job matches balance movements to our `deal_instruction` + `settlement` rows -- a mismatch is an exception.

Used by: planner (real cash constraint), settlement three-way match, treasurer dashboard.

Owner: the connectivity partner is a real decision -- see §3.4.

2.6 FX rates (spot + forward)

Today: an in-house `QuoteEngine` that generates plausible-looking rates deterministically.

Phase 2 source: market data providers.

Provider	Latency	Cost	Fit
Bloomberg B-PIPE / BPS	sub-second	GBP GBP GBP	Bank-grade; heavy contract
Refinitiv Elektron / RDP	sub-second	GBP GBP GBP	Same story
Xignite	seconds	GBP	Cloud-first, REST, best for us to start with
ExchangeRate-API / open feeds	minutes	free	Only OK for demos, not a real product

Recommendation: Xignite for v1 of the paid tier; keep the deterministic engine behind a `MarketDataProvider` port so tests never touch a real feed and the demo VM keeps working.

Handling: 1. On-demand fetch when a treasurer opens the ticket / hedge modal. 2. Cache in Redis for N seconds (5s for spot, longer for forwards). 3. Every real rate used in a hedge is persisted with the hedge -- a hedge decision made at 09:47 must show the rate that was on the screen at 09:47, not today's rate.

Failure mode: feed down -> advisor grey-boxes the "propose" button and shows "Market data unavailable -- refresh in a moment". Never quote stale.

2.7 Interest-rate benchmarks

Today: not modelled.

Phase 2 source: central bank publications (Bank of England, ECB, Federal Reserve) -- all free, all daily.

Handling: daily job pulls SONIA / EUR STR / SOFR / EFFR / TONA and writes to a new reference table. Deal rates are then quoted as `benchmark + spread` and stored as such -- separates the market movement from the credit spread.

2.8 Deals (proposed, approved, booked)

Today: treasurers create deals via `/deals` POST. This IS the real behaviour -- nothing changes.

Phase 2: the write surface stays the same. Optional additions:

- **Bulk load API** -- a customer with a pre-existing deal population imports its live book via a one-off endpoint that skips the six checks (`purpose='HISTORICAL_IMPORT'`), with an audit event flagging every imported row.
- **Order-management-system integration** -- some customers use a separate OMS. Add a `POST /deals` API-key path that lets the OMS propose deals; the six-check gate still runs.

2.9 Bank confirmations

Today: seeded MT320-shaped rows.

Phase 2 source: real SWIFT messages, same protocols as §2.5.

Handling: 1. Confirmations arrive in the same inbox as statements. 2. Parser identifies MT300/MT320/MT202 vs camt/pain messages. 3. Match to `deal` by reference + amount + counterparty; write `confirmation` row. 4. Three-way match (deal <-> confirmation <-> statement line) is already implemented -- it just starts running against real data.

Failure mode: unmatched confirmation -> sits in `exception_item` with cause = `CONFIRMATION_UNMATCHED`. This already exists.

2.10 Payment instructions (outbound)

Today: `POST /deals/{id}/instruct` writes to a table. Sends nothing.

Phase 2: 1. Same endpoint. 2. New adapter serialises the deal into MT202 or pain.001. 3. Signs the payload with the customer's HSM-held key (customer holds the key, not us -- we call out to their HSM via PKCS#11 or Azure Managed HSM). 4. Delivers via the same connectivity provider as inbound (§2.5). 5. Waits for the bank's acknowledgement (`ack` , `nack` , or `pain.002`). 6. Stores the ack in a new `bank_ack` table. The deal moves to `INSTRUCTED` only on a positive ack -- nothing else.

Failure mode: nack -> deal stays `APPROVED` , exception raised with the bank's reason code. Treasurer decides to re-instruct or amend.

Security: signing key never leaves the customer's HSM. We hold nothing that can create money movement.

2.11 Accounting postings

Today: `accrual` table written locally. No GL.

Phase 2 target: Oracle Fusion Cloud Financials -- General Ledger via the Treasury Register Connector. Non-Oracle ERPs are peer adapters behind the same connector interface but they are not the primary target.

How it flows out: 1. Every accrual / settlement / amendment write inserts an internal row (unchanged from today). 2. A poster job batches unsent rows every 15 minutes. 3. Poster calls the connector, which issues: `POST /fscmRestApi/resources/latest/journalEntries` with a payload naming the ledger, category `Treasury` , source `Treasury Register` , an `ExternalReference` we can reconcile against (`TR:accrual:<accrual_id>`), and two `JournalLines` (debit + credit). 4. Oracle returns `JeHeaderId` on success. Connector writes `gl_posting_ack` row linking `accrual_id` -> `oracle_je_header_id` . 5. A mismatch (posted internal row + no ack in 30 minutes) -> `exception_item.reason_code = GL_POSTING_STALE` + oncall alert.

Concrete payload:

```
 {
  "LedgerName": "Northgate UK Primary Ledger",
  "Category": "Treasury",
  "Source": "Treasury Register",
  "AccountingDate": "2026-09-21",
  "Description": "Interest accrual -- deal_id=deal_abc123",
  "ExternalReference": "TR:accrual:accr_xyz",
  "JournalLines": [
    {"AccountCombination": "01-100-4550-000-000", "EnteredCr": 425.68, "AccountedCr": 425.68, "Curre
    {"AccountCombination": "01-100-2500-000-000", "EnteredDr": 425.68, "AccountedDr": 425.68, "Curre
  ]
 }
```

The account combination string is customer-configured -- an account-mapping table (`gl_account_map`) maps our internal event kinds (`accrual` , `settlement_principal` , `settlement_interest` , `reversal_accrual` , `fx_gain` , `fx_loss`) to Oracle account combinations per legal entity.

Retry policy: retry on Oracle HTTP 429 (rate limit) + 503 (transient). Do NOT retry on 400/409 -- those are business errors (locked period, invalid account) that need human review. Escalate after 3 attempts on 500.

Failure mode: ERP down -> rows queue in our poster's outbox, exception surfaced on the compliance page. Nothing lost, nothing double-posted.

Idempotency: `ExternalReference` is unique per accrual -- Oracle's ADF Business Component checks this; a repeated POST returns the existing `JeHeaderId` rather than creating a duplicate journal. Belt and braces: our poster also refuses to send if the row already has a `gl_posting_ack`.

2.12 Users, roles and identity

Today: email + password -> bearer token. Activation links replace passwords. Sensible for a demo; not for enterprise.

Phase 2: 1. **OIDC via customer's IdP** -- Entra ID (default for corporate), Okta, Auth0. 2. Our app becomes an OIDC client. The IdP owns credentials + MFA + password rotation entirely. 3. Roles are read from an IdP group claim (e.g. `groups: ["Treasury.Analyst", "Treasury.HeadOfTreasury"]`) and mapped to our `membership.role`. 4. Every sign-in writes an `audit_event(action=user.signed_in)` exactly as today. 5. Bearer token remains the internal contract; only the front door changes.

Bridge behaviour: password-based sign-in stays available for `demo` tenants and disappears for real tenants (flag in `tenant`).

2.13 AI advisory activity -- the `ai_run` table

Today: Groq call happens, result flows through the deterministic engine, no record of what the AI saw or said.

Phase 2 -- closes Q-004:

New `ai_run` table (additive):

```
 id, tenant_id, occurred_at, endpoint (fx.narrate / advisory.rank / ...),
model, prompt_hash (SHA-256), prompt_text, response_text, response_ms,
tokens_prompt, tokens_completion, fallback_used (bool), subject_type,
subject_id, human_decision_id (nullable), outcome (accepted / rejected / n/a)
```

Handling: every AI call writes an `ai_run` row. When a human accepts an AI recommendation, the row is linked back. Compliance officer sees "the model recommended X on 12 May at 09:47; the treasurer chose X and booked it" -- this closes the loop.

Failure mode: AI fails -> deterministic fallback runs -> `ai_run.fallback_used=true`, no advisory quality drop.

---

3. Integration catalogue

Every external system, with the contract we sign against it.

3.1 Identity provider (SSO / MFA)

Every real customer signs in through their own IdP. Passwords in our DB retire on tenant flip.

Field	Value
Purpose	Sign-in + MFA + role mapping
Primary	**Microsoft Entra ID** (corporate default)

Field	Value
Alternatives	**Oracle IDCS / OCI Identity Domains** (native fit if customer is already Oracle Cloud, and it's the same tenant that hosts the Fusion Cloud service accounts for §3.3), **Okta**, **Auth0**
Protocol	**OIDC** -- Authorization Code + PKCE. Same code path for every IdP.
Endpoints we hit	`/authorize` (redirect), `/token` (exchange), `/jwks` (verify), `/userinfo` (optional profile)
Role mapping	IdP group claim (e.g. `groups: ["Treasury.Analyst","Treasury.HeadOfTreasury"]`) mapped to our `membership.role` via `sso_group_map` (new, additive)
MFA	Owned by the IdP entirely. We do not implement MFA -- that is the point of using the IdP.
Auth to us	ID token (JWT) verified against IdP JWKS; short-lived; we then mint our own bearer token for the app session
Failure mode	IdP unreachable -> new sign-ins refused with a clear "Sign in through your company account" message; existing bearer tokens continue to work until their TTL expires
Session TTL	Bearer token 1h (unchanged from today), refresh via silent OIDC on token expiry
Owner	Customer's IT / IAM team; we own the OIDC client configuration

3.2 Market data provider

Field	Value
Purpose	FX spot + forward + interest benchmarks
Primary (v1)	**Xignite** -- cloud-first, REST-first, good enough for a treasury product's cadence
Alternatives	**Refinitiv RDP** or **Bloomberg B-PIPE** for bank-grade latency, both GBP GBP GBP and heavy contract
Sample endpoint (Xignite)	`GET https://globalcurrencies.xignite.com/xGlobalCurre`
Forward rates	`GetForwardRate?Symbol=EURGBP&FwdTenor=3M`
Interest benchmarks (SONIA/EUR STR/SOFR)	Central bank APIs (free) OR Xignite `RatesLibrary`; nightly pull
Protocol	REST for pull. Websocket for streaming (not needed for a treasury product at this cadence)
Direction	Inbound
Cadence	On-demand fetch when a treasurer opens a ticket / hedge modal; 5s Redis cache for spot, 30s for forwards

Field	Value
Snapshot persistence	Every rate ACTUALLY USED in a hedge/deal is persisted with the write -- a decision made at 09:47 must show the rate that was on screen at 09:47, forever
Auth	API key in Key Vault, injected at startup
Failure mode	Feed down -> advisor greys out "propose" button, UI shows "Market data unavailable -- refresh in a moment". Never quote stale.
Owner	Us (procurement)

3.3 Oracle Fusion Cloud ERP -- via the Treasury Register Connector

Every ERP integration in this product runs through one component: the Treasury Register Connector. It is a first-party app (ADR-0004). It is **not** Oracle Integration Cloud (OIC). It is the only place in our stack that speaks Oracle Fusion Cloud, and every other service reads/writes Oracle only through it.

Why one connector, first-party

- Oracle Fusion Cloud REST endpoints have quirks -- pagination shape, quarterly deprecations, service-account rotation. Anywhere our services call Oracle directly, those quirks leak into a hundred places. The connector isolates them behind our own interface.
- Customers audit us; they audit our vendors. A first-party connector = one attestation surface, not two.
- We can version the connector's contract independently of Oracle's release cadence.
- The connector, being ours, is where we own retries, idempotency, error mapping, rate control and secret handling. None of those decisions leak into every consumer.

Oracle modules in scope

Oracle module	Business objects we	Business objects we
Financials -- General Ledger	`ledgers`, chart of accounts (for lookup)	`journalEntries` (deal accruals, reversals, settlements)
Financials -- Payables	`invoices` (AP), `payables` aging summary	--
Financials -- Receivables	`receivablesInvoices`, receivables aging	--
Cash Management	`cashManagementBankAccounts`, `bankStatements` (Oracle CM is a read-back mirror; primary statement flow is direct from banks -- §3.4)	`bankStatementLines`
Enterprise Performance Management (EPM Cloud)	Cash-forecast data forms in Planning + Budgeting Cloud	--
Cloud PPM (optional)	Project budgets (only when customer's forecast leans on PPM)	--

REST endpoint shape

Every call is the same URL shape:

```
https://<pod>-<region>.oraclecloud.com/fscmRestApi/resources/latest/<resource>
```

Sample query -- pull the AP invoices we need for the exposure snapshot:

```
GET /fscmRestApi/resources/latest/invoices
  ?q=BusinessUnit='Northgate UK Limited';
     InvoiceDate BETWEEN '2026-09-01' AND '2026-12-01'
  &fields=InvoiceNumber,SupplierName,SupplierNumber,InvoiceCurrency,
          InvoiceAmount,InvoiceDate,PaymentTermsName,BusinessUnit
  &limit=500&offset=0
Authorization: Bearer <token>
```

Response (elided):

```
{
 "items": [
   {
     "InvoiceNumber": "AP-2026-01123",
     "SupplierName": "Meridian Ltd",
     "InvoiceCurrency": "EUR",
     "InvoiceAmount": 250000.00,
     "InvoiceDate": "2026-09-15",
     "PaymentTermsName": "NET 45",
     "BusinessUnit": "Northgate UK Limited"
   }
 ],
 "count": 500, "hasMore": true, "offset": 0
}
```

The connector paginates by advancing `offset` until `hasMore=false`. It rate-limits to keep well under Oracle's 100 requests/second per tenant.

Auth flow -- Oracle IDCS / OCI Identity Domains

1. Customer's Oracle admin provisions a **service account** in their IDCS (or newer OCI Identity Domains) with: - Scope: `urn:opc:idm:__myscopes__` - Fusion Cloud application roles that permit read on AP / AR / Cash Management and write on GL journal entries only.
2. We obtain an OAuth2 bearer token via **client credentials grant**: `POST https://<idcs>.identity.oraclecloud.com/oauth2/v1/token grant_type=client_credentials&scope=<fusion-app-scope>`
3. Token TTL is 1 hour. Connector refreshes 5 minutes before expiry.
4. Token stored in memory only, plus a short-TTL Redis cache when multi-instance. Never persisted.
5. Blast radius on token compromise = 1 hour. Signing keys for money movement (§3.4) live in the customer's HSM, not here.

Data mapping -- the concrete "how"

Every mapping is per-customer configuration, stored in the connector, never inferred:

Our column	Comes from	Transformation
`currency_exposure.currency`	`Invoice.InvoiceCurrency`	as-is (ISO 4217)

Our column	Comes from	Transformation
`currency_exposure.gross_receivable_pence`	sum of `ReceivablesInvoice.InvoiceAmount` x 100	pence-integer round
`currency_exposure.gross_payable_pence`	sum of `Invoice.InvoiceAmount` x 100	pence-integer round
`currency_exposure.forecast_pence`	EPM data-form: `CashInflow - CashOutflow`, next 90d, by currency	pence integer
`currency_exposure.legal_entity`	`Invoice.BusinessUnit` -> `oracle_entity_mapping` table	derived, per-customer
`accrual` -> Oracle	`POST /journalEntries` with `AccountCombination` from `gl_account_map`	see §2.11

Two connector-owned tables are new, additive, and never touched by the treasury services: - `oracle_entity_mapping (business_unit_name, legal_entity_id)` - `gl_account_map (event_kind, legal_entity_id, account_combination)`

Both are edited via a dedicated admin surface at implementation time; changes go through the same four-eye control as any other config change.

Cadence + volume estimates

Flow	Cadence	Rough volume for a mid-cap treasurer
AP open-invoice read	Nightly 02:00 local	100–2,000 rows
AR open-invoice read	Nightly 02:00 local	100–2,000 rows
EPM cash forecast read	Nightly 02:00 local	30–90 rows (one per currency x time bucket)
GL journal write	Every 15 min	1–20 rows per batch
Bank statement read (Oracle CM fallback)	On-demand	Handful per legal entity

Volumes are well within Oracle Fusion Cloud's per-tenant limits -- the connector's own quota-management is the bottleneck we'd hit first at scale, not Oracle's.

Error handling

Oracle response	Connector behaviour
`2xx`	Ack, write `gl_posting_ack` where applicable
`429 Too Many Requests`	Retry with exponential back-off (up to 3 attempts)
`503 Service Unavailable`	Same

Oracle response	Connector behaviour
`400 Bad Request`	No retry -- business error (invalid account combination, locked period). Raise `exception_item` with the ORA-nnnnn code + Oracle's message. Human review.
`409 Conflict`	No retry -- duplicate `ExternalReference` typically means we already posted. Log, re-verify by GET, ack the existing journal.
`401 Unauthorized`	Force token refresh; retry once. Repeated 401 -> alert, escalate to IDCS/customer admin.
`500 Internal Server Error`	3 retries with back-off, then oncall alert + oncall runbook link

Failure modes worth naming

- **Oracle Fusion Cloud quarterly release breaks a field name.** Mitigation: connector version-pins the REST resource path; upgrades are scheduled, not surprises. We subscribe to Oracle's release notes.
- **Customer's Oracle Cloud pod maintenance window (usually weekend nights).** Mitigation: connector queues writes, alerts oncall, retries on the far side. Nothing lost.
- **Field deprecation (Oracle marks a field deprecated for 2 quarters).** Mitigation: connector logs deprecated-field usage; we migrate before Oracle removes.
- **Service account token stolen.** Blast radius 1h. Rotation via IDCS is zero-downtime.
- **Bad account-mapping configuration.** Postings fail on 400. `exception_item` names the mapping row that needs fixing.

Sequencing

The connector is a **Stage C** deliverable in §6 -- it depends on foundation (auth, secrets, observability) but unlocks the entire real-book stage.

Rollout order per customer: 1. Read-only: AP/AR/EPM extract -> `currency_exposure` snapshots. Prove exposure numbers reconcile against the customer's own reports (usually a spreadsheet the treasurer already keeps). 2. Read-only: Cash Management -> bank balances (only if bank connectivity §3.4 isn't in scope yet). 3. Write: GL journal postings, side-by-side with the customer's existing month-end process. Run for a full close cycle before cutover. 4. Cutover: the customer stops posting treasury journals manually.

What the connector is NOT

- Not Oracle Integration Cloud. Not a marketing sleight -- a genuine architectural choice.
- Not a general-purpose integration layer. It handles the treasury domain and nothing else.
- Not a place to add HR or CRM integrations. Those go through peer connectors or the customer's own iPaaS.
- Not a data warehouse. It transforms and forwards; it does not archive.

Non-Oracle ERPs

Peer adapters behind the same connector interface. Priorities:

ERP	Adapter approach	Priority
Oracle Fusion Cloud	REST + IDCS	**Primary -- Phase 2**
SAP S/4HANA	OData + BAPI	Second, when customer signs
NetSuite	REST + Saved Searches	Third

ERP	Adapter approach	Priority
Microsoft Dynamics 365 F&O	OData + Data Management API	Fourth
CSV fallback	SFTP inbox, standard schema	Available for any customer as a stopgap

Same `Connector.pull_exposure(as_of)` and `Connector.post_journal(entry)` interface; each adapter implements it. The treasury services never know which ERP the customer runs.

3.4 Bank connectivity

The other big integration alongside Oracle. The connector for this is **not** the Treasury Register Connector -- that one owns Oracle. Bank connectivity is a peer connector.

Field	Value
Purpose	Statements in, payments out, MT confirmations in
Options	Customer's own **SWIFT service bureau** (most common at mid-cap), **Fides**, **Kyriba Connect** (only if they already use Kyriba for something else), direct **SWIFTNet** (only banks and the very largest corporates)
Inbound protocols	**MT940** (end-of-day statement, legacy), **camt.053** (ISO 20022 statement, modern), **MT300 / MT320** (FX / MM confirmations), **camt.054** (credit/debit notification)
Outbound protocols	**pain.001** (payment initiation, ISO 20022, modern), **MT202** (MT-era cover payment), **MT103** (customer credit transfer)
Delivery	SFTP inbox (drop from bureau) OR SWIFTNet FileAct webhook
Direction	Bi-directional
Cadence	Daily statements 06:00 local, on-demand payments (queued and sent in a batch every 15 min), ad-hoc confirmations
Auth	Customer's BIC + SWIFT-network shared keys (managed by the bureau) OR bank host-to-host SFTP keys. **We never see the SWIFT keys.** Outbound payment signing happens in the customer's HSM (see below).
Signing outbound payments	pain.001 / MT202 payload is signed by the customer's HSM via PKCS#11 or Azure Managed HSM. We call out to sign, we get back a signed payload, we forward to the bureau. Our servers never hold a key that can create money movement.
Failure mode	Missing statement > 24h -> reconciliation blocks, exception raised; failed payment -> deal stays `APPROVED`, `exception_item.cause=BANK_ACK_MISSING`, oncall paged
Idempotency	Every outbound payment carries a UETR (Unique End-to-end Transaction Reference); repeat submissions with the same UETR are refused by the network

Field	Value
Owner	Customer's cash management / bank operations team

Oracle Cash Management interplay: If the customer already has statements loading into Oracle Fusion Cloud Cash Management, we can read them there instead of taking a duplicate feed from the bureau (§2.5). The Treasury Register Connector exposes `readBankStatement(as_of)` and the runtime chooses `bureau` or `oracle_cm` per customer configuration.

3.5 Rating agencies / KYC

Field	Value
Purpose	Counterparty rating + rating actions
Options	Refinitiv (primary), Bloomberg, direct S&P/Moody's/Fitch, ComplyAdvantage for KYC/AML
Protocol	REST / SFTP feed
Direction	Inbound
Cadence	Nightly refresh + intraday event push
Auth	Contract-per-customer or contract-per-vendor
Failure mode	Stale rating banner; never quote a rating without an `as_of`
Owner	Us

3.6 News / credit signals

Same shape as 3.5, cheaper. **GDELT free** as the first-line source, add a paid feed later.

3.7 Email delivery

Field	Value
Purpose	Invite emails, breach alerts, approval-request notifications
Options	AWS SES, Azure Communication Services, SendGrid, Postmark
Protocol	REST + SMTP
Direction	Outbound
Cadence	Per event
Auth	API key
Failure mode	Delivery fails -> retry with backoff, then dead-letter queue, then a Slack ping
Owner	Us

3.8 Secrets vault

Field	Value
Purpose	Every credential + API key + signing key
Options	Azure Key Vault (matches the VM host), AWS Secrets Manager, HashiCorp Vault
Protocol	Managed identity -> vault API
Direction	Inbound (we read)
Cadence	At startup + on-demand
Failure mode	Startup blocks; we do not run without secrets
Owner	Us

3.9 Observability

Field	Value
Purpose	Error tracking, structured logs, metrics, alerting
Options	Sentry (errors), Datadog / Azure Application Insights (metrics + logs), PagerDuty (alerts)
Protocol	Their SDKs
Direction	Outbound
Cadence	Continuous
Failure mode	Local structured logs never stop; observability is a nice-to-have degrade
Owner	Us

3.10 LLM host

Field	Value
Purpose	AI advisory narrator + planner narrator + FX briefing + news classifier (existing + new)
Options today	**Groq** -- fast, cheap, US-hosted. Fine for demos + customers with no data-residency constraint.
Enterprise alternatives	**Azure OpenAI** (EU/UK data residency, most corporate-friendly), **AWS Bedrock** (Anthropic Claude, Cohere; DPAs cover UK), **Oracle Cloud Infrastructure Generative AI** (native fit if customer is already OCI -- same tenant, same billing, same audit surface)
Protocol	HTTPS REST -- every provider has a compatible chat-completions surface behind an adapter

Field	Value
Direction	Outbound
Data residency	Customer-specific. UK bank -> OCI Gen AI in UK-South or Azure OpenAI in UK-South. Never prompt-to-US-endpoint without a signed DPA.
Model choice	Provider-agnostic prompt; the adapter picks the right model. Default: Groq `openai/gpt-oss-120b` (today), Azure `gpt-4o`, OCI `cohere.command-r-plus`, Bedrock `anthropic.claude-3-5-sonnet`.
Persistence	**All LLM activity is logged to `ai_run` (§2.13, closes Q-004).** Prompt, response, model, tokens, fallback flag, linked decision. Never a call without a row.
Failure mode	Deterministic fallback runs (already implemented). AI is never on a critical path -- the six-check gate does not consult it.
Owner	Us

3.11 Database (already partially done)

Postgres. See ADR-0018. Managed Postgres (Azure DB Flexible / RDS) for real deployments. Backups + PITR built into the service.

3.12 Cache / queue (new)

For rate-limit state (see roadmap P0-04 follow-up) and the outbound-posting queue. **Redis** is the default. Azure Cache for Redis / ElastiCache. Not needed until we go multi-instance.

---

4. Cross-cutting concerns

4.1 Data preservation

The rule is simple and absolute: **existing shapes are frozen.** Phase 2 adds tables and adds columns (always nullable), never renames, never widens a constraint that would drop data, never drops a column.

Every ADR from Phase 1 remains authoritative. Every audit event written yesterday must still be readable next year -- that is the whole promise of the compliance page.

Real customer data lives forever. The retention story is *archive to cold storage after 7 years*, not delete.

4.2 Multi-tenancy

Open -- Q-002. Two plausible shapes:

1. **Shared schema + `tenant_id` column** (current). Cheapest to run; needs row-level-security enforcement so a compliance officer at Customer A can never see Customer B.
2. **Schema-per-tenant.** Heavier but the isolation story writes itself.

Recommendation: stay with shared schema until the second customer signs, then decide. Postgres RLS is mature; SQLAlchemy pattern is well-known.

4.3 Data residency

UK bank -> data stays in the UK (Azure UK South, AWS eu-west-2). EU bank -> EU regions. US -> US. This is a contractual requirement, not a preference. All external services in §3 must have a region that matches the customer's jurisdiction.

4.4 Backup and disaster recovery

- **DB** -- managed Postgres -> point-in-time recovery, 35-day retention, automated snapshot.
- **File inbox (statements + confirmations)** -- retain the original raw file for 7 years in encrypted blob storage. Cheap and it saves you when a parser bug corrupts something silently.
- **Audit event** -- the immutable table. Never delete. Archive after 7 years.
- **RPO** -- 15 minutes. **RTO** -- 4 hours. Enough for a treasury system; not enough for a payments processor. Right target.

4.5 Security posture

- Bearer tokens -> OIDC (§3.1).
- Secrets -> vault (§3.8).
- CORS -> allow-list (P0-03 shipped).
- Rate limiting -> per-endpoint (P0-04 shipped).
- HTTPS enforced everywhere. HSTS. TLS 1.2+.
- SBOM published for each release. Dependabot on.
- Annual pen test.

4.6 Compliance regimes to satisfy

- **UK / EU** -- GDPR (data subject rights, retention, DPA). We are a data *processor* -- the customer is the controller.
- **Financial** -- depends on the customer's regulator. Bank customers = FCA/PRA-adjacent; corporate treasurers usually escape the direct regime but their auditor's expectations are as demanding.
- **ISO 27001** -- realistic 12-month target after the first paying customer.
- **SOC 2 Type II** -- 18-month target.

---

5. What NEW tables Phase 2 needs

All additive. Existing tables untouched.

Table	Purpose	Roughly-when
`cash_balance`	Daily bank balances per account per currency	With bank feed (§2.5)
`market_snapshot`	Cached FX + rate snapshots	With market data (§2.6)
`bank_ack`	Outbound payment acknowledgements	With payments (§2.10)
`gl_posting_ack`	ERP GL posting acknowledgements	With ERP posting (§2.11)
`sso_identity`	Maps IdP subject to `app_user`	With SSO (§2.12)
`ai_run`	AI activity log -- closes Q-004	With AI real activity (§2.13)
`integration_health`	Last-successful-run per integration	Cross-cutting

Table	Purpose	Roughly-when
`data_snapshot`	For point-in-time replay (P1-06)	Cross-cutting

---

6. Phase-2 sequencing recommendation

Build them in this order -- earlier ones unlock later ones and each is independently valuable.

Stage A -- foundation (weeks 1–3)

1. Postgres deployment (already done in code -- flip the env var).
2. Secrets vault (P0-06).
3. SSO -- Entra ID first, others behind the same OIDC adapter (P0-05).
4. Email delivery -- invites, breach alerts, approvals (P2-03).
5. Observability -- Sentry + structured logs (P3-02).

Gate to Stage B: a real user signs in via their IdP, causes an audit event, and everyone can see it in the compliance page.

Stage B -- market context (weeks 4–6)

6. Market data provider (§2.6).
7. Interest-rate benchmarks (§2.7).
8. Rating agency feed OR GDELT + news classifier (§2.2, §2.3).
9. `ai_run` table + Compliance AI tab (§2.13, Q-004).

Gate to Stage C: treasurer opens a ticket, sees a real rate, an AI advisor with a persisted history.

Stage C -- real book (weeks 7–10)

The Treasury Register Connector lands in this stage. It is one deliverable; every downstream flow in Stage C + Stage D reuses it.

10. **Treasury Register Connector v1 -- inbound only.** Oracle Fusion Cloud Financials AP + AR reads, EPM cash-forecast read, mapping tables (`oracle_entity_mapping` , `gl_account_map`), Oracle IDCS auth, retry + error-mapping. Emits `currency_exposure` snapshots (§2.4).
11. Bank statement feed (§2.5) -- either direct-from-bureau OR via Oracle Cash Management, connector chooses.
12. Confirmation matching against real MT / camt.053 (§2.9).
13. `cash_balance` in the planner (§2.5).

Gate to Stage D: a real deal, proposed by a real user, matched to a real confirmation, reconciled against a real statement, sitting against a real cash constraint. Nothing outbound yet.

Stage D -- outbound (weeks 11–14)

14. **Treasury Register Connector v2 -- outbound to Oracle GL.** `POST /journalEntries` with `ExternalReference` idempotency, `gl_posting_ack` reconciliation, retry policy per §2.11. Runs side-by-side with the customer's manual month-end journal-posting process for one close cycle before cutover.
15. **Payment instructions to bank** (§2.10) -- pain.001 + HSM signing + `bank_ack` loop.

Gate to product-ready: a deal booked in the register creates a signed pain.001 at the bank AND a journal entry in Oracle Fusion Cloud GL, both reconciled, both acked in `bank_ack` + `gl_posting_ack` , both

auditable through the compliance page's Deal Evidence tab (which already exists -- it just starts showing real acks).

---

7. What we still don't know

Feeds into `.context/open-questions.json`:

- **ERP: Oracle Fusion Cloud is the primary target** (Financials + Cash Management + EPM). Open: **which customer pod / region**, and **which specific Fusion release** we should test against (Oracle ships quarterly). Blocks §3.3 sizing.
- Which bank connectivity provider -- customer's own SWIFT service bureau, Fides, direct SWIFTNet? Blocks §3.4, §2.10.
- IdP: Entra ID (default) or **Oracle IDCS / OCI Identity Domains** (native fit if customer already runs Fusion Cloud through the same tenant)? Both fine; picking early avoids two client-config paths.
- Which market data vendor for v1 -- Xignite (recommended) vs. Refinitiv? Blocks §3.2 procurement.
- LLM host -- Groq (current), Azure OpenAI, **OCI Generative AI** (single-vendor story with Oracle-heavy customers), or AWS Bedrock? Depends on customer data residency. Blocks §3.10.
- Shared schema + RLS vs schema-per-tenant? Blocks second-customer readiness -- Q-002.
- Retention floor -- 7 years is a UK/EU norm; confirm with the customer's regulator. Blocks §4.1.

Every one of these is a customer conversation, not an engineering decision.

---

8. What this document is not

- **Not a sales pitch.** Every claim above must survive an audit.
- **Not a design spec for every table.** New tables are named and purpose'd; their columns will land in ADRs when built.
- **Not a commitment to every integration.** Some customers will use Excel exports for years. The order in §6 is what we *would* build; what we *will* build depends on the first paying customer's shape.
- **Not a rewrite.** The rule is preserved: same shapes, same tests, same six checks. Phase 2 changes where the data comes from, not what the register does with it.